

DATABASE AND SQL

Data: Raw facts and figures. We cannot make decisions on the basis of data

Database: A collection of data stored together to serve multiple applications is known as a database.

Advantages of Database:

1. Reduces the redundancy of data (unnecessary repetition/ duplication of data)
2. Maintains the data integrity (accuracy)
3. Allows the sharing of data
4. Maintains data consistency
5. Maintains data security. Only authorised persons can access the data from the database.
6. Automatic backup and recovery

DBMS: Database Management System refers to a tool that is responsible for storing, maintaining and utilising a database.

RDBMS: Relational Database Management System stores data in the form of tables (rows and columns)

Field: Set of characters that represents specific data elements. A collection of fields is called a record. A record can have fields of different data types. A collection of similar types of records is called a file.

Table: Collection of rows and columns that contains useful data/information. A table generally refers to the passive entity which is kept in a secondary storage device.

Relation: (collection of rows and columns) generally refers to an active entity on which we can perform various operations.

Tuples and Attributes: A row in a relation is called a tuple. A column in a relation is called an attribute. It is also termed as a field or data item.

Degree and Cardinality: Number of attributes in a relation is called the degree of a relation. Number of tuples in a relation is the cardinality of a relation.

Primary Key: Primary key is a key that can uniquely identify the records/tuples in a relation. This key can never be duplicated and NULL.

Foreign Key: Foreign Key is a key that is defined as a primary key in some other relation. This key is used to enforce referential integrity in RDBMS.

Candidate Key: Set of all attributes which can serve as a primary key in a relation.

Alternate Key: All the candidate keys other than the primary keys of a relation are alternate keys for a relation.

DBA: Database Administrator is a person (manager) that is responsible for defining the database schema, setting security features in the database, ensuring proper functioning of the data bases etc.

Structured Query Language: SQL is a non procedural language that is used to create, manipulate and process the databases(relations).

Characteristics of SQL

1. It is very easy to learn and use.
2. Large volumes of databases can be handled quite easily.
3. It is a non-procedural language. It means that we do not need to specify the procedures to accomplish a task but just to give a command to perform the activity.
4. SQL can be linked to most of other high level languages that makes it the first choice for the database programmers.

Processing Capabilities of SQL

1. Data Definition Language (DDL)

DDL contains commands that are used to create the tables, databases, indexes, views, sequences and synonyms etc.
Ex: Create table, create view, create index, alter table etc.

2. Data Manipulation Language (DML)

DML contains commands that can be used to manipulate the database objects and to query the databases for information retrieval.

Ex: Select, Insert, Delete, Update etc.

Data types of SQL

Just like any other programming language, the facility of defining data of various types is available in SQL also. Following are the most common data types of SQL.

1) INT: An integer value

2) FLOAT: Used to store a numeric value in a field/column. It may be decimal, integer or a real value. General syntax is

Float(n,d)

Where n = no. of digits, d = no. of digits to the right of the decimal point.

Ex:

pct number(5,2)

declares pct to be of type number of 5 digits with two digits to the right of the decimal point.

3) CHAR: Used to store character type data in a column. General syntax is

Char (size)

where size = maximum number of characters in a column. The CHAR type data can hold at most 255 characters.

Ex:

name char(25)

declares a data item name of type character of up to 25 size long.

4) VARCHAR: Used to store variable length alphanumeric data. General syntax is:

varchar(size)

where size = maximum number of characters in a column. The maximum allowed size is 2000 characters.

Ex: address varchar(50);

address is of type varchar of up to 50 characters long.

5) DATE: Date data type is used to store dates in columns. SQL supports the various date formats other than the standard DD-MON-YY.

Ex: dob date;

declares dob to be of type date.

Operators in SQL:

1. Arithmetic Operators +, -, *, /
2. Relational Operators =, <, >, <=, >=, <>
3. Logical Operators OR, AND, NOT

Constraints:

Constraints are the conditions that can be enforced on the attributes of a relation. The constraints come into play whenever we try to insert, delete or update a record in a relation.

1. NOT NULL: Not null ensures that we cannot leave a column as null. That is, a value has to be supplied for that column.

Ex: name varchar(25) not null;

2. UNIQUE: Unique constraint means that the values under that column are always unique.

Ex: Roll_no number(3) unique;

3. PRIMARY KEY: Primary key constraint means that a column can not have duplicate values and not even a null value.

Ex: Roll_no number(3) primary key;

The main difference between unique and primary key constraint is that a column specified as unique may have null value but primary key constraint does not allow null values in the column.

SQL Database Commands

Statement	Description	Syntax
CREATE DATABASE	Creates a database	create database <i>database_name</i> ;
USE DATABASE	Enters a database	use <i>database_name</i> ;
SHOW DATABASE	Displays all the databases available in a system	show databases;
DROP DATABASE	Deletes a database	drop database <i>database_name</i> ;

SQL Table Commands

SHOW TABLES	Displays all the tables in a database	show tables;
CREATE TABLE	Creates a table. It is a DDL type of command. The general syntax of creating a table is	create table <i>table_name</i> (<i>coll_name</i> [<i>data_type</i>] [<i>constraint(s)</i>], ... <i>coln_name</i> [<i>data_type</i>] [<i>constraint(s)</i>], [<i>table constraint(s)</i>]);
DESCRIBE TABLE	Describes a table structure	describe <i>table_name</i> ; or desc <i>table_name</i> ;
ALTER TABLE	Add column	alter table <i>table_name</i> add <i>column_name</i> [<i>data_type</i>];
	Drop column	alter table <i>table_name</i> drop column <i>column_name</i> ;
	Rename column	alter table <i>table_name</i> rename column <i>old_name</i> to <i>new_name</i> ;
	Alter/ Modify a datatype	alter table <i>table_name</i> alter column/ modify <i>column_name</i> [<i>data_type</i>];
DROP TABLE	Deletes a table	Drop table <i>table_name</i> ;
INSERT	Insert data into all the columns	insert into <i>table_name</i> values(<i>row1_values</i>),... (<i>rown_values</i>);
	Insert data into some specific columns	insert into <i>table_name</i> (<i>coll_name</i> , ... <i>coln_name</i>) values(<i>val1</i> , ... <i>valn</i>);
DELETE	Deletes a row based on specific instructions	delete from <i>table_name</i> where [<i>condition</i>];
SELECT	Displays the records of all columns	select * from <i>table_name</i> ;
	Displays the records of some specific columns	select <i>coll_name</i> , ... <i>coln_name</i> from <i>table_name</i> ;
ALIASING	Sets a temporary alternate name for a table or column. It only exists for the duration of that query	select <i>column_name</i> as ' <i>alias_colname</i> ' from <i>table_name</i> ; # <i>column name alias</i> select <i>column_name</i> from <i>table_name</i> as ' <i>alias_tablename</i> ' # <i>table name alias</i>
DISTINCT CLAUSE	Removes the duplicate values from the column	select distinct <i>column_name</i> from <i>table_name</i> ;

WHERE CLAUSE	Used to filter records. It is used to extract only those records that fulfil a specified condition.	<code>select * from table_name where [condition];</code>
IN	Used to extract data based on a list of values	<code>select * from table_name where column_name in (val1, ... valn);</code>
BETWEEN	Used to extract the data based on an upper and a lower value (Only for numeric values)	<code>select * from table_name where column_name between val1 and val2;</code>
ORDER BY	Used to sort the data in either ascending or descending order (Ascending by default)	<code>select * from table_name order by column_name asc/desc;</code>
IS NULL (NULL: When no value assigned to a column)	Used to extract the null values from a table	<code>select * from table_name where column_name is null;</code>
IS NOT NULL	The reverse of is null. Displays the not null values from a table	<code>select * from table_name where column_name is not null;</code>
LIKE	Extracts data on the basis of pattern matching. Patterns are matched using two special characters: % matches any substring _ matches any character	<code>select * from table_name where column_name like [condition ('m%', '_e%')];</code>
UPDATE	Used to update any column or value based on any condition	<code>update table_name set value where [condition];</code>
SUM()	Gives sum of values in a given column	<code>select sum(column_name) from table_name;</code>
AVG()	Computes the average of a given data	<code>select avg(distinct table_name) from table_name;</code> (distinct does not repeat values)
COUNT()	Counts the number of rows in a given column	<code>select count(table_name) from table_name;</code> (displays only NOT NULL values) <code>select count(*) from table_name;</code> (includes NULL values)
MAX()	Extracts maximum or highest value Date - latest date Char - Z to A Int/Float - Highest num	<code>select max(column_name) from table_name;</code>
MIN()	Extracts minimum or lowest value	<code>select min(column_name) from table_name;</code>
GROUP BY	Combines all the records that have identical values.	<code>select column_name(s) form table_name where [condition]</code> <code>group by [column_name(s)] order by column_name(s);</code>
HAVING	Places condition on groups (in contrast to where clause which only places condition on individual rows)	<code>select column_name(s) form table_name group by [column_name(s)] having [condition];</code>

JOINS: A join is a query that combines rows from more than one table.

1. CARTESIAN PRODUCT: (Unrestricted join of two tables) All possible concatenations are formed of all rows of both the tables. It returns $N1 * N2$ where
 $N1 = \text{No. of rows in table 1}$, $N2 = \text{No. of rows in table 2}$
 select *column_name(s)* from *table1, table2*;
2. EQUI JOIN: Columns are compared for equality
 select *column_name(s)* from *table1, table2*
 where *table1.column_common = table2.column_common*;
3. NATURAL JOIN: The join in which only one of the identical columns exists.
 select * from *table1* natural join *table2*;

CARTESIAN PRODUCT	EQUI JOIN	NATURAL JOIN
It joins the tables based on the column name specified in the ON clause explicitly. It returns only those rows that exist in both tables.	It joins the tables based on the equality or matching column values in the associated tables.	It joins the tables based on the same column names and their data types.
It returns all the attributes of both tables along with duplicate columns	It can return all attributes of both tables along with duplicate columns that match the join condition.	It always returns unique columns in the result set.